Md. Anowar Kabir

Dept. of ICT, MBSTU

Responsive Web Design (RWD):

Responsive web design (RWD) is a web design approach to make web pages render well on all screen sizes and resolutions while ensuring good usability. It is the way to design for a multi-device web. It uses techniques like flexible grids, responsive images, and CSS media queries to dynamically adjust the layout for good usability and appearance on any screen.

CSS Media Queries

CSS media queries allow you to apply styles based on the characteristics of a device or the environment displaying the web page.

CSS media queries are essential for creating responsive web pages.

The CSS `@media` rule is used to add media queries to your style sheet.

Media Query Syntax

A media query consists of an optional media-type and one or more media-features, which resolve to either true or false.

```
@media [not] media-type and (media-feature: value) and (media-feature: value) {
  /* CSS rules to apply */
}
```

Media Query Syntax

A media query consists of an optional media-type and one or more media-features, which resolve to either true or false.

```
@media [not] media-type and (media-feature: value) and (media-feature: value) {
  /* CSS rules to apply */
}
```

The media-type is optional. However, if you use **not**, you must also specify a media-type.

The result of a media query is true if the specified media-type matches the type of device, and all media-features are true. When a media query is true, the corresponding style rules are applied, following the normal cascading rules.

Meaning of the **not** and **and** keywords:

not: This optional keyword inverts the meaning of the entire media query.

and: This keyword combines a media-type and one or more media-features.

Media Queries Examples

Here, we use a media query to change the background-color of the body to lightgreen, if the width of the viewport is 720px, or wider:

index.html

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="style.css">
```

```
</head>


<body>
<h1>Resize the browser window to see the effect!</h1>

<p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Praesentium beatae suscipit, minima optio voluptate excepturi provident assumenda, incidunt ad eum quidem voluptatibus eius laborum cumque fugiat natus. Eius, assumenda fugiat.</p>
</body>
</html>
```

style.css

```
body {
  background-color: pink;
}

@media screen and (min-width:720px) {
  body {
    background-color: lightgreen;
  }
}
```

ct

CSS Performance Optimization

Optimizing CSS makes your website load faster and run more smoothly; which also results in a better user experience. Here are some tips for optimizing CSS:

1. Use Simple Selectors

Use simple selectors when possible. Complex selectors increase the parsing time.

Bad Example

```
body #navlist ul li a.button:hover {
  background-color: blue;
}
```

Better Example

```
.button:hover {
  background-color: blue;
}
```

2. Avoid Universal Selector for Styling

Avoid the universal selector (*) when not strictly necessary. The universal selector (*) affects every element and can slow down page rendering.

Example

```
* {
  margin: 0;
  padding: 0;
  font-size: 16px;
}
```

3. Avoid Inline Styles

Avoid inline styles when not necessary. Inline styles make your HTML heavier and are harder to manage.

Bad Example

```
<div style="color: red; font-size: 18px;">Hello</div>
<p style="color: blue; font-size: 16px;">Test</p>
```

4. Avoid @import

Avoid using @import for loading external CSS, as it delays stylesheet loading.

Add external CSS with the <link> tag in the head section, so it loads before the page is rendered.

Example

```
<link rel="stylesheet" href="style.css">
```

5. Use Shorthand Properties

Use shorthand properties when possible. It saves space and is faster to parse.

Example

```
/* Long version */
margin-top: 10px;
margin-right: 20px;
margin-bottom: 10px;
margin-left: 20px;

/* Shorthand version */
margin: 10px 20px;
```

6. Cut Down Unnecessary Animations

A high number of animations and large animations require more processing power to handle, which degrades performance. So, remove unnecessary animations.

7. Use Properties that Not Cause Repaint of Animations

Animation performance relies also on what properties you are animating. Some properties (like width, height, left, top), trigger a layout recalculation when animated, and should be avoided. If possible, use animation properties that do not cause repaint, like transforms, opacity and filter.

8. Combine and Minify CSS

Use one CSS file when possible, and remove spaces and comments to reduce file size. You can use tools like:

- CSS Minifier
- PostCSS
- Online compressors